A Sampling-and-Filtering Framework with Offline Policy Exploration for Certified Loop Invariant Generation

Anonymous
Anonymous Institution
anonymous@example.com

Abstract

Loop invariant generation is a central hard problem in program verification. Existing LLM-based approaches rely on extensive SMT solving, complex workflows, and the most powerful closed-source models, yet still struggle with this task—which demands logical reasoning, program comprehension, and mastery of formal annotation syntax. We present SAM2INV, a system that trains a small, locally deployable model to generate high-quality loop invariants through offline knowledge distillation with verifier feedback. To overcome training-data scarcity, we first design LoopFactory, a probabilistic DSL constrained by semantic templates that synthesises structured numeric loop programs at arbitrary scale. We then build a three-stage sampling-and-filtering pipeline: N parallel LLM calls are unioned into a single candidate pool and filtered through (1) an ACSL syntax filter, (2) an execution-trace semantic filter, and (3) Houdini formal pruning via Frama-C/WP, producing certified invariant annotations. Rather than applying reinforcement learning—whose binary verifier reward is too sparse, reward-hackable, and ineffective for weak base models—we adopt a distillation strategy: a lightweight teacher model explores multiple candidates per program during data preparation; the verification pipeline filters them into chosen and stage-labelled rejected examples; and the student is fine-tuned via SFT followed by DPO, transferring the teacher's exploration benefit with no human annotation or online reward model.

1 Introduction

Loop invariant generation is a central hard problem in program verification. Traditional approaches [4, 5, 8, 9, 22] can each handle specific families of loops but generalise poorly beyond their pre-designed search spaces, and often yield limited effectiveness at prohibitive analysis time. The emergence of large language models (LLMs) has brought fresh perspectives to this problem, and several recent works have explored LLM-based invariant generation [3, 15, 25]. However, these approaches typically rely on extensive SMT solving, static analysis pipelines, and complex multi-round workflows, and they universally employ the most powerful closed-source models available at the time. This reliance itself reveals that loop invariant generation remains an exceptionally challenging task even for LLMs: it demands not only *logical reasoning* and *program comprehension*, but also mastery of the specific formal syntax and semantics of the target annotation language. Training a *locally deployable* model that generates high-quality loop invariants without relying on massive API token budgets or heavyweight external tools—and at substantially lower runtime cost—is therefore of considerable practical value. Moreover, loop invariant generation differs fundamentally from typical code generation or mathematical reasoning benchmarks: it better reflects a model's integrated mathematical and programming ability in a realistic formal-methods setting. We therefore also aim to investigate whether small, open-weight models can achieve strong performance on this task.

Before training can begin, we must first overcome the scarcity of formal training data. Existing invariant benchmarks contain only a few dozen programs [2], which is far too few for fine-tuning and raises severe data-leakage concerns. To address both issues, we design LoopFactory, a probabilistic domain-specific language (DSL) constrained by semantic templates, which synthesises well-formed numeric loop programs at arbitrary scale (Section 5).

For loop invariant correctness we do possess a verifiable feedback signal: an SMT solver can decide whether a candidate satisfies the inductiveness conditions. However, this signal is a binary pass/fail verdict—too sparse to serve directly as a reinforcement learning reward. More concretely, relying solely on the binary signal is susceptible to *reward hacking*: trivially weak annotations (e.g. `loop invariant true ;`) can formally pass the checker, earning full reward while capturing no mathematical structure. Designing a richer reward through hand-crafted heuristics or a trained reward model introduces excessive *human-prior bias*. Furthermore, methods such as GRPO [6] require the model to roll out multiple candidate answers per input and update the policy based on relative rewards; loop invariant generation is hard enough that a weak base model rarely produces a satisfactory answer within a limited sampling budget, rendering online exploration ineffective.

Based on these considerations, we adopt a *knowledge distillation* approach. As demonstrated by the DeepSeek-R1 distilled models [6], reasoning patterns of a strong teacher can be transferred to small models via distillation more effectively than through RL on the small model directly; we believe distillation better enables a small model to expand its reasoning repertoire and explore novel solution paths for this task. For the teacher we choose a relatively lightweight model (GPT-5-nano), avoiding an excessive student–teacher capability gap while keeping token cost tractable. We move GRPO-style exploration into the *data-preparation* phase rather than distilling the teacher's single-best answer directly. In this process we embed the verifier's feedback signal: all candidate

answers sampled from the teacher are combined, deduplicated, and filtered through a three-stage verification pipeline; the surviving certified annotation is assembled as the *chosen* example, while every rejected candidate becomes a *rejected* example. The student is then trained in two stages—first supervised fine-tuning (SFT) on the chosen examples, then direct preference optimisation (DPO) on the chosen/rejected pairs—transferring the teacher's exploration benefit to the student, grounded entirely in formal verification, with no human annotation or online reward model.

Contributions. In summary, this paper makes three contributions:

(1) **LoopFactory**, a probabilistic DSL for numerical loop synthesis that overcomes training-data scarcity and data leakage by generating programs with tunable complexity and known semantic structure (Section 5).
(2) **A three-stage sampling-and-filtering pipeline** that issues N parallel LLM calls, unions all candidates, and filters them through an ACSL syntax filter, an execution-trace semantic filter, and Houdini formal pruning via Frama-C/WP—certifying every surviving invariant (Section 4).
(3) **An offline knowledge-distillation strategy** that embeds verifier feedback into the data-preparation step, producing SFT and DPO training data from teacher-model exploration, and fine-tunes a small locally deployable student model without online RL (Section 4.4).

The remainder of this paper is organised as follows. Section 2 surveys related work. Section 3 gives a high-level overview of the pipeline. Section 4 details each component. Section 5 describes LoopFactory. Section 6 presents experimental evaluation. Section 7 concludes.

2 Related Work

Classical Loop Invariant Generation. Invariant synthesis methods span a spectrum from fully static to fully dynamic. Abstract interpretation [5] computes fixed-point over-approximations in abstract domains (intervals, octagons, polyhedra) and underpins industrial analysers, but is restricted to the expressiveness of the chosen domain. Template-based constraint solving [4, 28] fixes a parametric invariant form and searches for satisfying coefficients via LP or SDP; the approach is complete within the template family but cannot discover structure outside it. The Houdini algorithm [9] starts from a large candidate set and iteratively removes non-inductive conjuncts; we adopt it as the third stage of our filter pipeline. Daikon [8] and DIG [22] mine likely invariants from execution traces; like us they use traces as evidence, but require a separate verifier pass to guarantee inductiveness. All these methods are confined to their pre-designed search spaces and generalise poorly when the invariant falls outside.

LLM-Based Loop Invariant Generation. Recent work has applied LLMs to invariant synthesis. Chakraborty and Lahiri [3] show that GPT-4 can generate invariants for simple C programs with a verification feedback loop. Pei et al. [25] study code-trained LLMs producing Hoare-style annotations and find that iterative refinement with verifier feedback is essential. Kamath et al. [15] propose a guess-and-check framework combining LLM generation with bounded model checking. In the proof-assistant ecosystem, LLMs suggest tactics [13, 14] and translate informal statements into formal proofs [18, 26, 30]. A common thread across these works is heavy reliance on external tool chains (SMT solvers, static analysers, multi-round repair workflows) and the most powerful closed-source models available, without exploring whether a small, locally deployed model can be trained for this task. SAM2INV differs in two respects: a systematic three-stage filter pipeline that pre-screens candidates before formal verification, and an offline distillation strategy that transfers the capability of a teacher model to a small student without requiring online exploration.

Knowledge Distillation vs. Reinforcement Learning for Reasoning. Reinforcement learning from human feedback (RLHF) [24] and direct preference optimisation (DPO) [27] are standard alignment techniques for LLMs. Replacing human preferences with verifiable rewards from automated oracles—reinforcement learning with verifiable rewards (RLVR)—has produced large gains in mathematical reasoning: DeepSeek-R1 [6] and Tulu 3 [17] train on verifier-generated rewards with no human annotation, while execution-guided methods [11, 19] use test-case signals for code generation. However, the DeepSeek-R1 distilled models [6] demonstrate that reasoning patterns of a strong teacher can be transferred to small models via distillation more effectively than through RL on the small model directly, because distillation directly transfers successful solution trajectories rather than requiring the student to rediscover them through sparse reward signals. In the loop invariant domain, the binary verifier reward is particularly problematic: it is sparse (a weak model rarely produces correct annotations), reward-hackable (trivial annotations pass weak postconditions), and non-attributable (the correct annotation is a conjunction assembled from multiple calls). Our approach inherits the RLVR spirit but moves exploration entirely offline into the data-preparation phase, using verifier feedback to construct ground-truth preference pairs for SFT+DPO distillation—avoiding the instability of online RL for weak base models.

Training Data Synthesis for Program Verification. Existing invariant synthesis benchmarks—the NLA suite from SV-COMP [2], the linear benchmark set [29]—contain only a few dozen hand-crafted programs, insufficient for both systematic evaluation and large-scale training. Grammar-based fuzzing [12] and probabilistic program synthesis [23] generate programs at scale but without the semantic constraints needed to ensure well-defined invariant structure. In the theorem-proving domain, AlphaCode [20] applies large-scale sampling and filtering for competitive programming, a paradigm related to ours, but deductive verification with

SAM2INV Pipeline

❶ **Smart Sampling** — Execute the program with tiered inputs; collect execution traces recording variable values at every loop iteration.

❷ **Parallel LLM Generation** — Issue N parallel LLM calls with diverse prompts and temperatures; parse and union all candidate invariants into a single pool.

❸ **Stage 1 – ACSL Syntax Filter** — Reject candidates that violate ACSL syntax rules (unsupported quantifiers, forbidden operators, etc.).

❹ **Stage 2 – Execution-Trace Semantic Filter** — Discard candidates falsified by any observed loop state without invoking the verifier.

❺ **Stage 3 – Houdini Formal Pruning** — Iteratively remove non-inductive invariants via Frama-C/WP; the surviving set is *certified*.

❻ **Knowledge Distillation** — Record verified candidates as SFT chosen examples; pair rejected candidates with chosen as DPO preference pairs, labelled by rejection stage. Output annotated C code (or report failure).

Figure 1: The SAM2INV sampling-and-filtering pipeline. Steps ❶–❷ form Phase I (Sampling & Generation); steps ❸–❺ form Phase II (Three-Stage Filtering); step ❻ constitutes Phase III (Certification & Knowledge Distillation). Houdini pruning in step ❺ is guaranteed to terminate.

Frama-C/WP does not admit step-by-step proof search—the verifier accepts or rejects a complete annotation in one shot, making the filtering pipeline critical. LoopFactory addresses the data gap by generating structured numeric loop programs with tunable complexity via semantic core templates, supplying both training inputs and controlled benchmarks for complexity ablation.

3 System Overview

Figure 1 illustrates the end-to-end SAM2INV pipeline. The system takes as input a C function containing a `while` loop, a precondition (`requires`), and a postcondition (`assert`). It outputs the same function annotated with ACSL loop invariants and a `loop assigns` clause such that Frama-C/WP can fully verify the postcondition, together with SFT and DPO training records for offline policy fine-tuning.

The pipeline operates in three broad phases.

Phase I: Sampling and Generation (Steps ❶–❷). The target program is compiled and executed under a set of carefully chosen inputs produced by the *smart tiered sampler* (Section 4.1). Execution traces—recording variable values at every loop iteration—are collected and formatted into structured text that exposes the mathematical relationships maintained by the loop. Multiple LLM instances are then queried in parallel, each receiving a prompt assembled from the source code, execution traces, and a system prompt encoding ACSL rules and an assertion-driven synthesis strategy. Prompt diversity is achieved by varying the template and the sampling temperature. The union of all parsed responses forms the initial candidate pool.

Phase II: Three-Stage Filtering (Steps ❸–❺). The candidate pool is subjected to three progressively more expensive filters. The *ACSL syntax filter* checks each candidate against a set of structural rules—no unsupported quantifiers, no forbidden operators, correct use of `\at` expressions—and discards violators immediately. The *execution-trace semantic filter* evaluates each surviving candidate against every recorded loop state; any candidate falsified by at least one observed state is discarded without invoking the verifier. Finally, *Houdini formal pruning* passes the pre-screened set to Frama-C/WP, iteratively removing candidates whose verification conditions fail until the remaining set is fully inductive. Because at least one candidate is removed per Houdini iteration, termination is guaranteed (Theorem 4.1). Every candidate that exits Phase II is formally certified by Frama-C/WP.

Phase III: Certification and Knowledge Distillation (Step ❻). The certified invariant set is written back into the source file as the verified output. Simultaneously, the pipeline produces training data for offline knowledge distillation. Candidates accepted through all three filter stages are the *chosen* response for supervised fine-tuning (SFT). Candidates rejected at any stage are paired with the chosen annotation to form *direct preference optimisation* (DPO) preference pairs, labelled by rejection stage. This design moves GRPO-style exploration offline: the teacher model explores during pipeline execution; the formal verifier acts as the quality filter; and the student learns from the resulting structured preference data—with no online rollouts, no human annotation, and no trained reward model required.

Running Example. Consider the following program that computes the cube of n:

```
/*@ requires a>=n && n==0; */
int main1(int a, int n){
  int x, y, z;
  x=0; y=1; z=6;
  while(n <= a){
    n=n+1; x=x+y; y=y+z; z=z+6;
  }
  /*@ assert (n==a+1) && (y==3*n*n+3*n+1)
            && (x==n*n*n) && (z==6*n+6); */
}
```

The smart sampler executes this program with $a \in \{0, 1, \ldots, 10\}$ and collects traces such as:

iter	n	x	y
0	0	0	1
1	1	1	7
2	2	8	25

The parallel LLM generation step produces a pool of candidate invariants. Ill-formed candidates (e.g., those using exponentiation `^`) are eliminated by the syntax filter; candidates that evaluate to `false` on any observed row are eliminated

by the trace filter. Houdini pruning then confirms the four inductive invariants $x = n^3$, $y = 3n^2+3n+1$, $z = 6n+6$, and $n \leq a+1$, and Frama-C/WP verifies the postcondition in a single pass. The verified invariants are stored as SFT data; every rejected candidate is paired with this verified set to form a DPO training record.

4 Method

We now describe each component of SAM2INV in detail, following the pipeline order introduced in Section 3.

4.1 Smart Dynamic Sampling

The purpose of dynamic sampling is to collect execution traces that expose the mathematical relationships maintained by the loop. Naive random sampling often produces inputs that are either too simple (all zeros, which collapse polynomial terms) or too large (causing integer overflow or time-outs). We therefore adopt a *tiered* sampling strategy: parameter values are partitioned into four tiers—special corner values $\{0, \pm 1\}$, small positives, medium values, and large values—and input tuples are generated in phases of increasing tier complexity, with a biased random fill for any remaining quota. Each input is executed, and the variable state at every loop iteration is recorded. To keep prompts within LLM context limits, we retain only the first and last k iterations per run (default k=3) and group traces into at most G groups (default G=10). Full details of the tier definitions, phased Cartesian sampling algorithm, and trace formatting are given in Appendix F.

4.2 Parallel LLM Generation

Each LLM prompt is assembled from three components: a *system prompt* encoding the task definition and allowed ACSL constructs, the *source code* of the target function, and *execution traces* formatted as structured variable–value tables. The system prompt encodes an *assertion-driven* strategy: decompose each postcondition conjunct as an invariant candidate, weaken the loop guard to obtain a boundary invariant, and preserve unmodified function parameters via `\at(v, Pre)`. Full prompts are reproduced in Appendix D.

To maximise the probability of finding the complete invariant set, we issue N parallel LLM calls (default N=5) with temperatures sampled from $\{0.8, 1.0, 1.1, 1.2\}$. Each call j produces a candidate set C_j; these sets are *unioned*, not voted upon:

$$C_{\text{pool}} = \bigcup_{j=1}^{N} C_j \quad \text{(whitespace-normalised deduplication).}$$

The union-then-filter design is motivated by a structural property of loop invariants: the correct annotation is a *conjunction* of multiple clauses, and different clauses tend to be discovered by different calls. After filtering, the certified output C^* is a post-hoc collective result, not a selection from any individual response—a *non-attributability* property formalised in Appendix B.5.

4.3 Three-Stage Filter Pipeline

The candidate pool is filtered through three progressively more expensive stages; the design principle is to discard as many incorrect candidates as cheaply as possible before invoking the next stage.

Stage 1: ACSL Syntax Filter. Each candidate is checked against structural ACSL rules—no unsupported quantifiers, no forbidden operators, correct use of `\at` expressions, balanced brackets—and discarded immediately on any violation. This filter runs in microseconds per candidate. The full rule set is listed in Appendix E.

Stage 2: Execution-Trace Semantic Filter. Each surviving candidate is evaluated as a boolean expression over every recorded loop state. Any candidate falsified by at least one observed state is discarded without invoking Frama-C. This eliminates candidates with grossly incorrect arithmetic relationships cheaply, though it cannot guarantee inductiveness since the traces are finite.

Stage 3: Houdini Formal Pruning. The pre-screened candidates are submitted to Frama-C/WP. We apply the Houdini algorithm [9]: starting from the full candidate set, each iteration removes every candidate whose verification condition fails, repeating until no candidate is removed or the set is empty.

Theorem 4.1 (Termination). *Houdini pruning terminates in at most $|C|$ iterations, since each iteration removes at least one candidate.*

The output C^* is the maximal inductive subset of the pre-screened candidates. Every invariant in C^* is formally certified by Frama-C/WP. If C^* is non-empty but insufficient to prove the postcondition, the system may optionally enter a brief repair loop (at most K=3 iterations) feeding verification errors back to the LLM. The complete per-loop algorithm is given in Appendix B.

Both improvements—union assembly and lightweight pre-screening—are *intrinsic to the pipeline itself*: they hold for any base LLM, before any fine-tuning. The offline knowledge distillation described next is a separate improvement layer built on top of this already-strong baseline.

4.4 Offline Knowledge Distillation

Why not reinforcement learning. Online RL with binary verifier feedback faces four structural obstacles in this domain: sparse reward (a weak model rarely produces a correct annotation within a practical rollout budget); reward hacking (trivial annotations such as `loop invariant true;` formally pass weak postconditions); human-prior bias (a hand-crafted or trained reward model scales poorly); and non-attributability (the correct conjunction is assembled post-hoc from N calls, so per-call binary scoring mislabels essential contributions; see Appendix B.5).

Knowledge distillation via verifier feedback. We instead adopt offline *knowledge distillation* from a teacher model, but

depart from standard distillation in a key respect: rather than imitating the teacher's raw outputs, we move GRPO-style exploration offline and embed the formal verifier's signal into data preparation. The teacher is a capable yet lightweight model—chosen so that the student–teacher gap remains bridgeable and per-program token cost stays tractable.

For each training program, the teacher generates N candidate annotations at diverse temperatures; all outputs are pooled and passed through the three-stage filter. The surviving certified set C^* is the *chosen* response. Every candidate eliminated at stage $k \in \{1, 2, 3\}$ becomes a *rejected* response paired with C^* to form a DPO record, with stage-dependent loss weight ($w_1 \leq w_2 \leq w_3$) reflecting that later-stage rejections are harder and more informative contrasts. The SFT record is generated alongside each successful run. The preference labels are ground-truth correct by construction: no human annotation or trained reward model is needed. Formal definitions and the stage-stratified loss are given in Appendix C.

Two-stage fine-tuning. The student is first trained with SFT on chosen examples, then aligned with DPO on preference pairs. This two-stage recipe transfers the teacher's exploration benefit to the student without online rollouts. The filter pipeline certifies every output regardless of model quality—a weaker model yields a smaller C^* but never an uncertified one—so the system remains sound throughout the improvement cycle.

5 Numerical Loop Synthesis

To overcome the scarcity of annotated numeric loop programs, we design LoopFactory, a probabilistic DSL that synthesises well-formed C loop programs at arbitrary scale. A program has the form $\mathbf{Init}; \mathbf{Loop}^+$, where loops may be sequential or nested; structural complexity is freely tunable via fuel parameters governing the number of variables, loops, assignments, and conditional blocks per program. Because uniform sampling from the grammar overwhelmingly produces divergent or semantically vacuous programs, we constrain generation with *semantic core templates*: each template fixes a termination-guaranteed update skeleton and a known invariant shape, while leaving variable names and constants to probabilistic instantiation. The 60 templates span eight proof-obligation families—linear conservation, monotone accumulation, proportional stride, modular counting, state-machine transitions, division/GCD, multiplicative redistribution, and polynomial recurrence—ensuring that the corpus covers the full range of arithmetic loop patterns an invariant-generation policy must handle. Using LoopFactory we synthesise 2,000 programs; 1,900 are used for training and 100 are held out as a synthetic test set. Full DSL definition, template descriptions, and generation examples are given in Appendix A.

6 Experiments

7 Conclusion

We have presented SAM2INV, a system for certified loop invariant generation built on three contributions.

LoopFactory, our probabilistic DSL, overcomes the scarcity of annotated loop programs by synthesising structured numeric loops with freely tunable complexity, supplying both training inputs and controlled benchmarks.

The *three-stage sampling-and-filtering pipeline* assembles a diverse candidate pool through smart tiered sampling and parallel LLM generation, then eliminates incorrect candidates via an ACSL syntax filter, an execution-trace semantic filter, and Houdini formal pruning. Every surviving invariant is formally certified by Frama-C/WP. The pipeline delivers substantial improvements even before fine-tuning: union assembly discovers conjuncts no single call would produce, and the lightweight filters prevent parse failures from reaching the verifier.

The *offline knowledge distillation* strategy moves GRPO-style exploration into data preparation: a teacher model generates candidates, the verifier filters them, and the resulting chosen/rejected pairs form SFT and DPO training data grounded entirely in formal verification—no human annotation, no online reward model, and no reward hacking. Two-stage SFT + DPO fine-tuning transfers the teacher's exploration benefit to a smaller student model while the pipeline guarantees soundness throughout the improvement cycle.

Several directions remain for future work. Most directly, systematic fine-tuning experiments should measure how the success rate improves as the student model is updated over successive training rounds. Extending the system to heap-manipulating programs, where invariants must express separation-logic properties, is a natural next step; our codebase already partially supports Coq/VST verification. Finally, integrating symbolic techniques—such as abstract interpretation for initial bounds or interpolation for strengthening failing candidates—could yield a hybrid symbolic–neural architecture with stronger guarantees outside the current system's success region.

A LoopFactory: Full DSL Description

A.1 DSL Grammar

A LoopFactory program consists of an initialisation block followed by one or more loops:

$$\mathbf{Prog} ::= \mathbf{Init}\ ;\ \mathbf{Loop}^+ \qquad \mathbf{Loop} ::= \texttt{while}\ (g)\ \{\ \mathbf{Body}\ \}$$

$$\mathbf{Body} ::= \big(\ v := e \ \mid\ \texttt{if}\ (g)\ \{S\}\ [\texttt{else}\ \{S\}]\ \mid\ \mathbf{Loop}\ \big)^*$$

Variables $\mathcal{V}$ are partitioned into immutable *parameters* $\mathcal{P}$ and writable *state variables* $\mathcal{L}$. Body expressions have the form $v\ \texttt{op}\ x$ with $\texttt{op} \in \{+, -, \times, \mathrm{mod}\}$ (/ additionally permitted for nonlinear programs); the guard g is a linear comparison or the constant `1` (for `while(1)` with `break`). **Init** assigns every variable in $\mathcal{L}$ exactly once from expressions over $\mathcal{P}$.

A.2 Multi-Loop and Nesting

The grammar supports *sequential* multi-loop programs (**Loop**$^+$), where later loops share state computed by earlier ones, and *nested* loops (a **Loop** inside **Body**), bounded by a maximum depth $D_{\max}$. Each loop is generated independently with its own counter and guard, drawing state variables from the shared pool within budget m.

A.3 Complexity-Controlling Fuel Parameters

Structural complexity is governed by fuel parameters (Table 1), each sampled uniformly from a configurable $[min, max]$ range. By adjusting these bounds the generator produces programs ranging from trivial single-variable counters to multi-loop programs with rich arithmetic dependencies—without modifying the generator or template library. This makes LoopFactory suitable both for training-data generation and as a controlled benchmark tool for evaluating how invariant-generation difficulty scales with program complexity.

Table 1: Fuel parameters controlling per-program complexity.

Parameter	Controls	Default range
m	Total state variables	≤ 6
$[p_{\min}, p_{\max}]$	Immutable parameters	$[1, 2]$
$[n_{\min}, n_{\max}]$	Sequential loops	$[1, 1]$
$[k_{\min}, k_{\max}]$	Assignments per loop body	$[1, 6]$
$[f_{\min}, f_{\max}]$	Conditional blocks per loop	$[0, 3]$
$D_{\max}$	Maximum nesting depth	1

A.4 Semantic Core Templates

Uniform grammar sampling overwhelmingly produces divergent or semantically vacuous programs. Each *semantic core template* suppresses both failure modes by fixing three aspects of the loop:

- **Counter and guard**: the progression mode (increment, decrement, doubling, ...) is matched to the guard, guaranteeing termination.
- **Core variable updates**: a fixed set of assignments maintains the semantic invariant; these variables are write-protected during subsequent sampling steps.
- **Invariant-consistent initialisation**: initial values are set so the invariant holds from the first iteration.

With probability $\pi_{\text{core}} = 0.88$ a loop is seeded by a template; the remaining 12% are assembled freely. After the template installs its skeleton, an extension phase fills the remaining assignment budget with freely sampled expressions over non-core variables.

A.5 Semantic Families

The 60 templates span eight families, each with a distinct proof-obligation shape:

- *Linear conservation* ($a+b = N$): variables exchange a fixed quantity each iteration; their linear combination is constant.
- *Monotone accumulation* ($s \mathrel{+}= f(k)$): a running sum grows monotonically; the invariant relates it to the counter via a closed form (e.g. $2s = k(k-1)$, $s = k^2$).
- *Proportional stride* ($j = c \cdot i$): two counters advance at a fixed ratio, requiring a proportionality invariant.
- *Modular counting* ($c \equiv t \pmod T$): a variable cycles through a residue class; the invariant involves modular arithmetic.
- *State-machine transitions*: discrete phase variables drive piecewise updates; the invariant is a disjunction over reachable phase combinations.
- *Division algorithm / GCD* ($a = q \cdot d + r$, $0 \leq r < d$): Euclidean reduction or quotient–remainder decomposition.
- *Multiplicative redistribution* ($z + x \cdot y = a_0 b_0$; or $z \leq 2^k$): a product is re-expressed across variables as they are scaled.
- *Polynomial recurrence*: coupled variables satisfy a quadratic or higher-degree relation maintained by finite-difference updates (e.g. $z \cdot w \geq p^2$).

A.6 Corpus

Using LoopFactory we synthesise 2,000 numeric loop programs spanning all eight semantic families. Of these, 1,900 serve as the training corpus annotated by the SAM2INV pipeline; the remaining 100 are held out as a synthetic test set for evaluating invariant-generation success in this controlled loop space.

B Detailed Invariant Generation Algorithm

B.1 Data Structures

The pipeline maintains the following state for each outer loop index i:

- **record**$_i$: the structured descriptor extracted from static analysis, containing the loop guard, assignments (transition relation), initial variable values, invariable (unmodified) parameters, and function parameter names.
- $\mathcal{T}_i$: the execution trace set, a list of variable-state snapshots collected by the smart sampler.
- C_i: the live candidate pool (a set of ACSL expression strings).
- C_i^*: the certified invariant set (output).

B.2 Houdini Pruning Algorithm

B.3 Complete Per-Loop Algorithm

B.4 Role of Static Analysis

Before Phase 1, the sampler's `LoopAnalysis` module performs a *lightweight static pre-pass* on the input C file to populate

Algorithm: Houdini Pruning
Input: Candidate set C, verifier $\mathcal{V}$
Output: Maximal inductive subset $C^* \subseteq C$

```
C* ← C
repeat
    R ← V(C*)                          // verify each invariant
    F ← {c ∈ C* | R(c) = fail}
    C* ← C* \ F
until F = ∅ or C* = ∅
return C*
```

Figure 2: Houdini-style pruning algorithm used in Stage 3 of the filter pipeline.

Algorithm: InvariantGenerator per loop i
Input: Source code P, record $\mathbf{r}$, traces $\mathcal{T}$, LLM $\mathcal{M}$, verifier $\mathcal{V}$
Output: Annotated code P^* or $\perp$

```
Phase 1 — Generation:
P_tmpl ← InsertTemplate(P, i)          // PLACE_HOLDER annotations
(σ, ℓ) ← PreparePrompt(r, T)           // (system, user) prompts
for j = 1, ..., N in parallel do
    R_j ← M(P_tmpl, σ, ℓ, τ_j)         // diverse temperature
    C_j ← ExtractInvariants(R_j)
end for
C ← ⋃_j C_j                            // union of all candidates

Phase 2a — Heuristic ACSL Syntax Gate:
C ← {c ∈ C | SyntaxFilter(c, r) = pass}

Phase 2b — Execution-Trace Semantic Gate:
C ← {c ∈ C | ∀s ∈ T : eval(c, s) = true}

Phase 2c — Conflict Detection:
C ← RemoveConflicts(C)

Phase 2d — Merge:
C_merged ← Dedup(C)                    // whitespace-normalised

Phase 3 — Houdini Formal Pruning:
C* ← Houdini(C_merged, V)              // see Algorithm 2
if C* = ∅ then return ⊥

Phase 4 — Postcondition Check & Optional Repair:
if V reports assert satisfied then
    return Annotate(P, C*)
else run ≤ K repair iterations (feed error to M)
return Annotate(P, C*) or ⊥
```

Figure 3: Full per-loop invariant generation algorithm.

record$_i$. This is *not* symbolic execution in the strict sense; it is a source-level analysis that:

(1) Identifies the loop guard (transition relation) by reading the `condition` field recorded in the JSON analysis file produced by the Frama-C preprocessing step.
(2) Extracts the set of variables that are never assigned inside the loop body (*unchanged variables*), which become candidates for $\backslash\texttt{at}(v, \mathrm{Pre})$ invariants.
(3) Infers conservative integer bounds $[\ell_v, u_v]$ for each parameter via `LoopBoundAnalyzer`, which are used to constrain the smart sampler's input domain.
(4) Builds the **record** structure that encodes loop content, transition expression, pre-condition, and the names of function parameters (needed to gate `\at` usage in the syntax filter).

The Frama-C/WP oracle is *only* invoked during the Houdini pruning stage (Phase 3) and never during candidate generation or lightweight filtering.

B.5 Non-Attributability of the Certified Output

We formalise the key structural property stated informally in Section 4.2.

Definition B.1 (Attribution). Given N candidate sets $C_1, \ldots, C_N$ and a certified output C^*, we say C^* is *attributable to call j* if $C^* \subseteq C_j$.

Proposition B.2 (Non-attributability is typical). *Suppose the correct annotation for a loop requires a conjunction of $m \geq 2$ invariant clauses $c_1, \ldots, c_m$. If each clause c_i is discovered independently by each of the N calls with probability $q \in (0,1)$, then the probability that* all *clauses appear in any single call C_j is q^m, while the probability that all clauses appear in the union $C_{pool} = \bigcup_j C_j$ is $1-(1-q)^{mN}$. For $q = 0.6$, $m = 4$, $N = 5$: single-call coverage $= 0.6^4 \approx 0.13$; union coverage $= 1 - 0.4^{20} \approx 1.00$.*

Proof. Follows directly from independence and the complement rule for union of events. □

Proposition B.2 shows that the gap between single-call and union coverage grows exponentially with the number of required conjuncts m. In practice, real loop invariants for arithmetic programs require $m \in [2, 6]$ conjuncts (boundary invariants, postcondition conjuncts, auxiliary equalities), making single-call attribution unlikely in typical cases.

Corollary B.3. *Under the conditions of Proposition B.2 with $m \geq 2$ and $q < 1$, the expected fraction of per-call outputs C_j for which $C^* \subseteq C_j$ is $q^m < 1$. Assigning a positive reward to calls satisfying $C^* \subseteq C_j$ and a negative reward to others would on average penalise $(1-q^m) > 0$ fraction of calls that may have contributed essential clauses to the union.*

This corollary shows that any *per-call* binary scoring scheme — including GRPO — systematically mislabels calls that contribute essential clauses to the final answer, degrading the training signal for exactly the behaviours (diverse conjunct coverage) that make the union effective.

C DPO Data Construction: Formal Specification

C.1 Chosen Response Construction

Given a program P and the certified invariant set C^* for loop i, the chosen annotated program is:

$$P^+ = \texttt{Annotate}(P,\ i,\ C^*)$$

where `Annotate` inserts the ACSL block

```
/*@ loop invariant c_1; ...loop invariant c_k;
        loop assigns vars; */
```

immediately before the `while` statement of loop i. The `loop assigns` clause is derived from the set of variables assigned anywhere in the loop body. P^+ is guaranteed to satisfy Frama-C/WP because it survived Stage 3 (Houdini pruning); no additional verification is needed for the training record.

C.2 Rejected Response Construction and Stage Labelling

For each $r \in C_{\text{pool}} \setminus C^*$, we construct:

$$P_r^- = \texttt{Annotate}(P,\ i,\ \{r\})$$

and record the stage $k(r) \in \{1, 2, 3\}$ at which r was eliminated:

- $k(r) = 1$: r failed the ACSL syntax gate (Appendix E).
- $k(r) = 2$: r passed syntax but was falsified by some trace state $\mathbf{s} \in \mathcal{T}$, i.e. $\texttt{eval}(r, \mathbf{s}) = \text{false}$.
- $k(r) = 3$: r passed syntax and trace filters but was removed by Houdini because Frama-C/WP could not verify its establishment or preservation condition.

The full DPO training record for program P, loop i, and rejected expression r is the tuple:

$$\mathcal{D}(P, i, r) = \big(P,\ P^+,\ P_r^-,\ k(r)\big).$$

C.3 Stage-Stratified Loss Weighting

Standard DPO minimises the loss:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(P^+ \mid P)}{\pi_{\text{ref}}(P^+ \mid P)} - \beta \log \frac{\pi_\theta(P_r^- \mid P)}{\pi_{\text{ref}}(P_r^- \mid P)}\right)\right].$$

We extend this with stage-dependent weights w_k:

$$\mathcal{L}_{\text{S-DPO}} = -\mathbb{E}\left[w_{k(r)} \cdot \log \sigma\left(\beta \log \frac{\pi_\theta(P^+ \mid P)}{\pi_{\text{ref}}(P^+ \mid P)} - \beta \log \frac{\pi_\theta(P_r^- \mid P)}{\pi_{\text{ref}}(P_r^- \mid P)}\right)\right]$$

The rationale for $w_1 \leq w_2 \leq w_3$ is that Stage-3 rejections are the hardest preference distinctions: the rejected expression is consistent with all observed program states and was eliminated only by a formal inductiveness check, making the margin between P^+ and P_r^- the smallest and the training signal the most informative. Concretely, we use $w_1 = 0.5,\ w_2 = 1.0,\ w_3 = 2.0$ as default weights, reflecting the increasing difficulty of the corresponding rejection judgements.

C.4 SFT Records

Alongside DPO pairs, each successful pipeline run produces an SFT record $(P,\ P^+)$. SFT pre-training warms up the model towards the format and style of ACSL annotations before DPO alignment, following the standard two-phase fine-tuning recipe [24]. Because P^+ is formally verified, SFT on these records teaches the model what a *correct and informative* annotation looks like, providing a positive learning signal that complements the contrastive DPO signal.

D Prompts Used in Each Stage

D.1 System Prompt

The system prompt is loaded verbatim from `prompts/system_prompt.txt` and prepended to every LLM call. Its full content is reproduced below.

```
You are a formal verification expert specializing in loop
invariant synthesis for Frama-C with the WP plugin.

## TASK
Given a C function with a loop and a postcondition
(/*@ assert ... */), generate ACSL loop annotations
(invariants, assigns) that enable Frama-C/WP to verify
the assertion.

## CORE STRATEGY: Assertion-Driven Invariant Synthesis
1. Decompose the assertion: each conjunct of the assert
   (split on &&) is a candidate loop invariant.
2. Add boundary invariants: weaken the loop guard.
   - while(i < n) needs: i <= n
   - while(n <= a) needs: n <= a+1
3. Preserve parameter values: if a parameter is unmodified,
   assert: loop invariant x == \at(x, Pre);
4. Verify sufficiency: check that (invariants AND NOT guard)
   implies the postcondition.

## FORBIDDEN CONSTRUCTS
- \forall, \exists  (quantifiers not supported)
- predicate, logic, axiomatic, lemma
- Ternary ? :
- \at(local_var, Pre)  (\at only on function parameters)
- ^ for exponentiation  (write x*x*x for x cubed)

## ALLOWED OPERATORS
Comparison: ==, !=, <, <=, >, >=
Logical:    &&, ||, !, ==>, <==>
Arithmetic: +, -, *, /, %

## OUTPUT
Return ONLY the annotated C function. Do not include
explanations.
```

D.2 User Prompt (Loop Context Block)

The user prompt is assembled by `PromptFormatter` and passed as the second message in the conversation. It consists of two parts.

Part 1 / Loop context.

```
### Loop Context: ###

1. Loop Context
  A. Pre-Condition (Before Loop Entry): `<precondition>`
  B. Loop Transition Relation: `<transition_expr>`
  C. Loop Snippet:
```c
<loop_code>
```


2. Execution Traces
   Each trace shows the full sequence of conditional
   evaluations, step by step.

   WARNING: CRITICAL TEMPORAL SEMANTICS:
   - Each state is labeled as
     'BEFORE loop body execution' or 'AFTER loop terminates'
```

```
  - Loop invariants must hold at the START of each iteration
    (BEFORE body executes)
  ...
```

Part 2 — Execution traces. For each sampled input group (up to G=5), the formatter emits a trace block:

```
[TRACE 1]
    [ n==0 && x==0 && y==1 && z==6 ]
        (BEFORE loop starts) ->
    [ n==1 && x==1 && y==7 && z==12 ]
        (BEFORE iteration 1 body executes) ->
    [ n==2 && x==8 && y==25 && z==18 ]
        (AFTER loop terminates)
```

The temporal labels (`BEFORE iteration` k `body executes`) are essential: they tell the LLM that an invariant must hold at loop entry of each iteration, not at loop exit, which is a common source of error in naive prompting. Initial values use `\at`(v, Pre) notation so the model can directly reference them in `loop invariant` lines.

Diversity via temperature. Each of the N parallel LLM calls uses an independently sampled temperature τ_j. In the default configuration, the primary call uses $\tau_1 = 1.0$ and subsequent calls are drawn uniformly from $\{0.8, 1.0, 1.1, 1.2\}$, ensuring that the candidate pool covers both conservative and more exploratory invariant conjectures.

E Heuristic ACSL Syntax Gate: Implementation Details

The syntax gate (Stage 1, Section 4.3) runs before any program execution or Frama-C invocation. It is implemented in `unified_filter.py` as a rule-based checker that operates directly on ACSL expression strings.

E.1 Rejection Rules

Each candidate invariant expression is checked against the following ordered rules (any single failure causes rejection):

(1) **Trivial/placeholder tokens.** Expressions equal to `true`, `false`, `...`, or containing only punctuation are rejected immediately.

(2) **Non-printable or non-ASCII characters.** Frama-C requires printable ASCII (U+0020–U+007E). Characters such as the Unicode inequality signs $\leq$, $\geq$, $\neq$, or control characters (e.g. BEL, U+0007 occasionally emitted by LLMs) cause rejection. The error message instructs the model to use `<=`, `!=`.

(3) **Nested `loop invariant` keyword.** When a LLM fills in a `PLACE_HOLDER` slot with a complete `loop invariant expr;` line instead of a bare expression, Frama-C cannot parse the annotation and enters a pathological parsing state. A regex check (`/\bloop\sinvariant\b/i`) catches and rejects these.

(4) **Bracket balance.** Unmatched parentheses, square brackets, or braces are rejected, with the position of the first mismatch reported in the error message.

(5) **Forbidden quantifiers.** Presence of `\forall` or `\exists` triggers rejection; Frama-C/WP does not support quantifiers in loop invariant annotations.

(6) **Forbidden ACSL definitions.** The keywords `predicate`, `inductive`, `logic`, `axiomatic`, `lemma` may not appear in invariant expressions.

(7) **Forbidden ACSL math operators.** `\product`, `\sum`, `\min`, `\max` are rejected (not supported by the WP plugin for loop invariants).

(8) **Ternary operator.** The `? :` construct is not valid ACSL.

(9) **Undefined variable names.** Each identifier in the expression is compared against *known_vars*, the union of local variables, parameters, and ACSL built-ins extracted from **record**. Unknown identifiers are rejected with a diagnostic listing the candidates.

(10) **`\at` scope check.** Expressions of the form `\at`(v, Pre) are permitted only when $v \in \mathcal{P}$ (function parameters). Usage on local variables is rejected, since local variables have no pre-state binding in Frama-C/WP.

(11) **Exponentiation (`^` or `\pow`).** A final safety sweep removes any surviving candidates containing these tokens, which represent bitwise XOR or undefined ACSL functions respectively.

E.2 Post-Merge Sanitisation

After the union of N candidate pools is formed (Algorithm 3, line 11), an additional normalisation pass is applied:

- `\at`(v, Pre) for local variable v is rewritten to plain v (conservative safe fallback).
- Any invariant re-introducing `^` or `\pow` is silently dropped.

This sanitisation step prevents the Houdini stage from encountering annotation parse errors that would stall the Frama-C subprocess.

F Smart Dynamic Sampling: Implementation Details

F.1 Overview

The smart sampler (`smart_sampler.py`) generates concrete integer inputs for the program's function parameters. Its design goal is to produce a small but informationally dense set of execution traces that expose the polynomial relationships maintained by the loop, while avoiding:

- All-zero inputs (which collapse polynomial terms and prevent coefficient identification).
- Excessively large inputs (which cause integer overflow or time-outs during dynamic execution).

F.2 Tiered Value Generation

For each parameter v with domain $[\ell_v, u_v]$ inferred by `LoopBoundAnalyzer`, the sampler constructs four tiers:

$\mathcal{T}_0(v) = \{0, 1, -1\} \cap [\ell_v, u_v]$	(special corner values)
$\mathcal{T}_1(v) = \{2, \ldots, 10\} \cap [\ell_v, u_v]$	(small positive integers)
$\mathcal{T}_{1'}(v) = \{-2, \ldots, -10\} \cap [\ell_v, u_v]$	(small negatives, if enabled)
$\mathcal{T}_2(v) \subseteq \{11, \ldots, 50\}$	(5 randomly drawn medium values)
$\mathcal{T}_3(v) \subseteq \{51, \ldots, 100\}$	(3 randomly drawn large values)

F.3 Phased Cartesian Sampling

Inputs are generated in four phases. Each phase fills part of the quota of at most $S_{\max}$ samples (default $S_{\max} = 20$). Already-seen input tuples are deduplicated via a hash set.

(1) **Phase 1 (corner cases).** Generate all tuples in $\prod_v \mathcal{T}_0(v)$. For p parameters each with up to 3 special values, this produces at most 3^p tuples (e.g. 9 for $p = 2$). These inputs are ideal for fitting degree-0 and degree-1 polynomial terms.

(2) **Phase 2 (mixed tier 0+1).** Enumerate combinations where each coordinate is drawn from $\mathcal{T}_0(v) \cup \mathcal{T}_1(v)$. Combinations are ordered by total tier index $\sum_v \text{tier}(v_j)$, so simpler combinations are generated first.

(3) **Phase 3 (all tiers).** Extend to include $\mathcal{T}_2$ and $\mathcal{T}_3$, again prioritised by total tier index. This covers medium-range inputs needed to disambiguate quadratic from linear relationships.

(4) **Phase 4 (biased random fill).** If the quota is not yet reached, fill with random tuples where each coordinate is drawn with probability 0.8 from $[0, 20]$ and 0.2 from $[21, 100]$. This phase ensures coverage of edge cases not reached by the structured phases.

F.4 Trace Collection and Formatting

Each accepted input tuple is passed to `DynamicExecutorConfigurable`, which instruments the C source with `printf` statements recording all variable values at the top of each loop iteration. The collected trace for a single input is a sequence of variable-assignment strings (e.g. `(n == 2) * (x == 8) * (y == 25) * (z == 18)`). These are parsed and formatted by `PromptFormatter` into the temporal-labelled trace blocks shown in Appendix D.

To prevent context-window overflow, the formatter applies two limits:

- At most $G = 5$ trace groups are included per LLM call.
- Within each group, at most $k_{\max} = 10$ iteration snapshots are kept; if a loop runs for more than $k_{\max}$ iterations, the first $\lfloor k_{\max}/2 \rfloor$ and last $\lceil k_{\max}/2 \rceil$ snapshots are retained, and the truncation is noted explicitly in the prompt.

These limits ensure that the total prompt length stays within the LLM's context window even for programs with deeply nested loops or large parameter ranges.

References

[1] Josh Berdine, Cristiano Calcagno, and Peter W. O'Hearn. 2005. Smallfoot: Modular Automatic Assertion Checking with Separation Logic. In *Proc. Int. Symp. on Formal Methods for Components and Objects (FMCO)*. Springer, 115–137.

[2] Dirk Beyer. 2023. Competition on Software Verification (SV-COMP). https://sv-comp.sosy-lab.org/.

[3] Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. 2024. Towards Neural Synthesis for SMT-Assisted Proof-Oriented Programming. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

[4] Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. 2003. Linear Invariant Generation Using Non-linear Constraint Solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*. Springer, 420–432.

[5] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*. 238–252.

[6] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025).

[7] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. Can Large Language Models Write Specifications?. In *Proc. ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE)*.

[8] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon System for Dynamic Detection of Likely Invariants. In *Science of Computer Programming*, Vol. 69. 35–45.

[9] Cormac Flanagan and K. Rustan M. Leino. 2001. Houdini, an Annotation Assistant for ESC/Java. In *Proc. Int. Symp. on Formal Methods Europe (FME)*. Springer, 500–517.

[10] Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. 2016. Learning Invariants Using Decision Trees and Implication Counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*. 499–512.

[11] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen Carbonell, et al. 2024. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. *arXiv preprint arXiv:2410.02089* (2024).

[12] Patrice Godefroid, Adam Kiezun, and Michael Y. Levin. 2008. Grammar-based Whitebox Fuzzing. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*. 206–215.

[13] Jesse Michael Han, Floris Rabe, Yuhuai Wang, Christian Szegedy, and Christian Szegedy. 2022. Proof Artifact Co-Training. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

[14] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2023. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

[15] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. 2023. Finding Inductive Loop Invariants using Large Language Models. *arXiv preprint arXiv:2311.07948*.

[16] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. 2015. Frama-C: A Software Analysis Perspective. In *Proc. Int. Conf. on Software Engineering and Formal Methods (SEFM)*. Springer, 233–247.

[17] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengding Huang, Hamish Ivison, Faeze Brahman, Lester James Validad Miranda, Valentina Pyatkin, et al. 2024. TÜLU 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv preprint arXiv:2411.15124* (2024).

[18] Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Aurelien Rodriguez, Amaury Hayat, Thibaut Lavril, Gabriel Ebner, and Xavier Martinet. 2022. HyperTree Proof Search for Neural Theorem Proving. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*.

[19] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning.

In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*.

[20] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-Level Code Generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097.

[21] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

[22] ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. 2012. Using Dynamic Analysis to Discover Polynomial and Array Invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*. 1076–1086.

[23] Aditya V. Nori, Sherjil Ozair, Sriram K. Rajamani, and Deepak Viber. 2015. Efficient Synthesis of Probabilistic Programs. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*. 208–217.

[24] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*.

[25] Kexin Pei, David Biber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *Proc. Int. Conf. on Machine Learning (ICML)*.

[26] Sorin Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. 2023. Formal Mathematics Statement Curriculum Learning. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

[27] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model Is Secretly a Reward Model. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*.

[28] Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. 2004. Non-linear Loop Invariant Generation Using Gröbner Bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*. 318–329.

[29] Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. 2018. Learning Loop Invariants for Program Verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*. 7751–7762.

[30] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Christian Szegedy, and Christian Szegedy. 2022. Autoformalization with Large Language Models. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*.